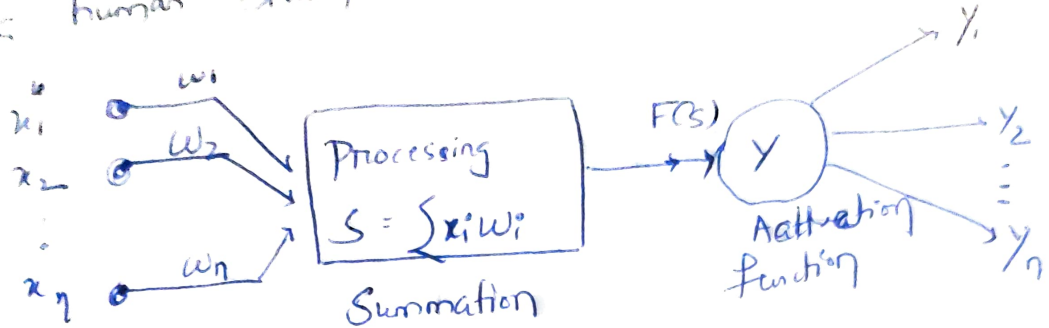
mimic human brain.



(Artifical Neural Network)

Codebasics

⇒ Computer Vision → amazon go – (Shopping)

- ↳ google lens
- ↳ photos (Google) – Classification based on (Person / , fruits ,)
- ↳ bank Check Identification (Bank of America)
- ↳ Autonomous car (Tesla)
- ↳ Agriculture (farming area Scans)

Image size: 1920 × 1080 × 3 (RGB)

(I/p) first layer neuron 1920 × 1080 × 3 ~ 6 million

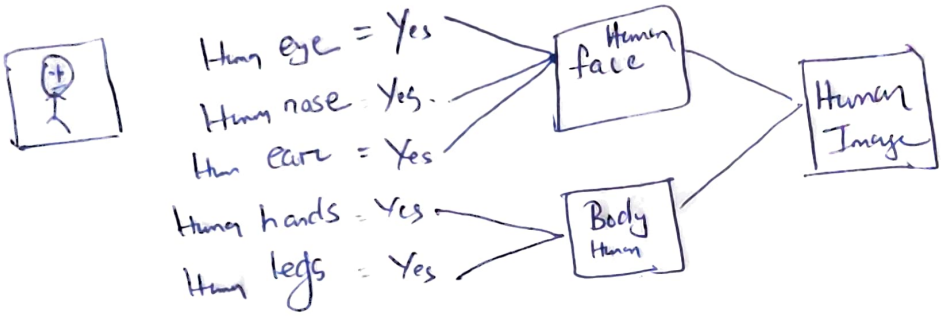
HL neuron = Let's 4 million

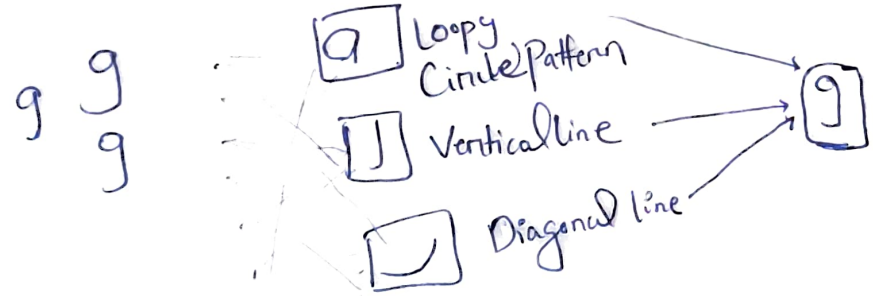
weights between I/p & HL = 6 million × 4 million

= 24 Trillion weights calculated.

→ Disadvantage of ANN for Image Classification

1. Too much Computation
2. Sensitive to location of an Object in an Image







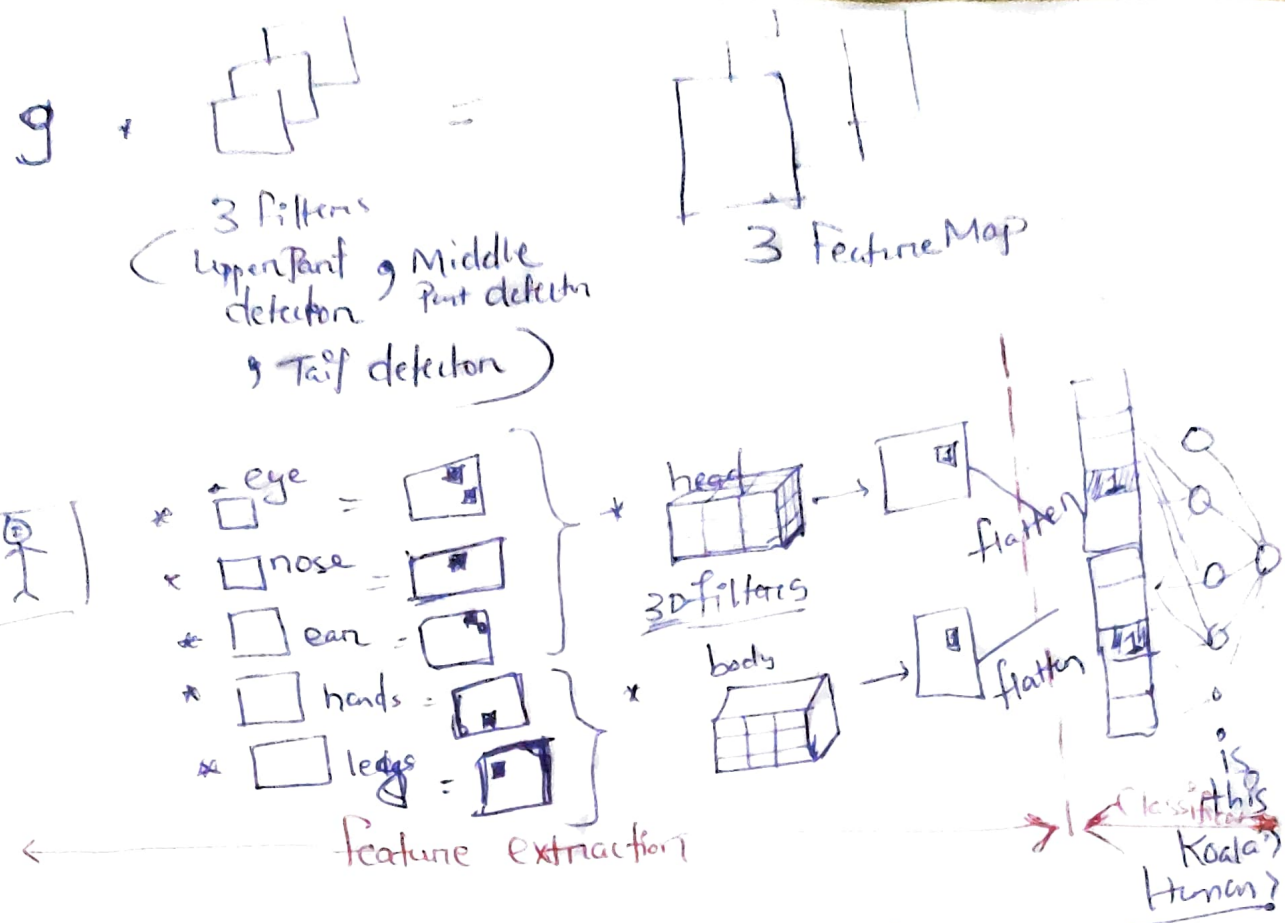
-1+1+1-1-1-1-1-1+1+1 → 1/9 → -0.11

Convolution Operation

Stride = 1

Location Invariant; if detects Koalas eyes in any location of the image.

as filter is moving throughout the Image.



→ Pooling is used to reduce the size of Image / dimension of Image as to optimize Computational cost.



(2×2 filter with Stride = 2) [[max pooling]]

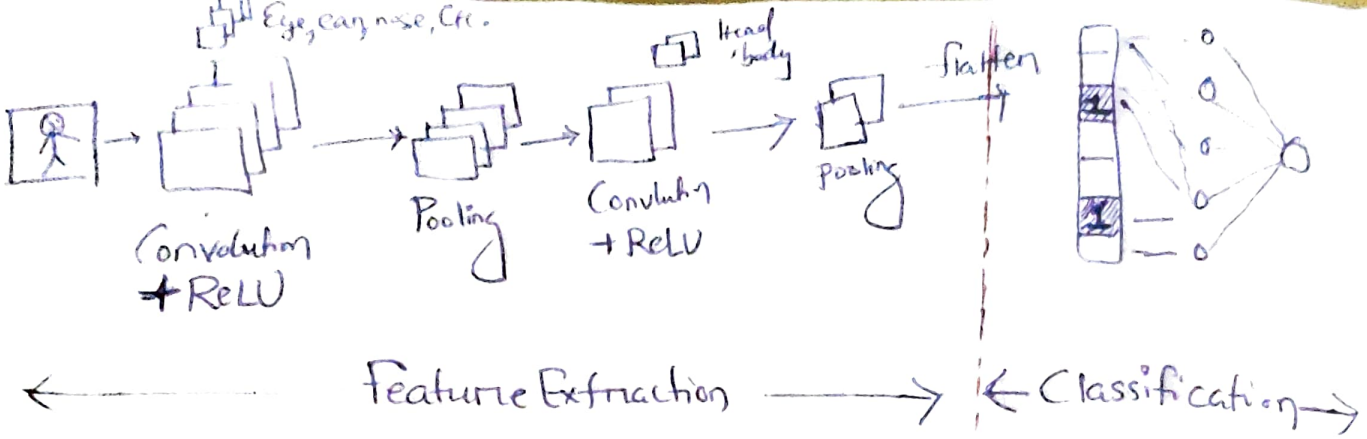
Saving Computations by reduce Size / dimension

(Maxpolling + Convolution) ——— helps to detect Position invariants detections

↳ Average Pooling

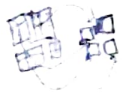
Benefits of Pooling :

↳ Reduce Dimensions & computation
↳ Reduce Overfitting as there is less Parameters
↳ Model is tolerant towards Variations/distortion as max pooling choose imp feature & filters noise.



Convolution

→ ~~Reduce Sparsity~~ Connections Sparsity (Not like Dense/Conv)
→ reduces overfitting
→ Conv + Pooling gives location Invariant feature detection
→ Parameter Sharing



ReLU

→ Introduces Nonlinearity
→ Speeds up training, faster to Compute



Pooling

→ Reduce dimensions & Computations
→ Reduce Overfitting
→ make the Model tolerant towards small distortion and/ Variations.

→ **Rotation & Thickness :**





→ CNN itself can't handle Rotation & thickness / scale.
→ You need training Samples (like Rotated, thick, thin etc.)
→ If you don't have such training Samples then use data augmentation methods to generate new rotated/scaled samples from existing training samples.

* The filters will be learned during the training of Neural Network
→ Neural Network learns its filters automatically.

→ Image Classification using CNN (CIFAR10 dataset)

60000 32x32 colour Images in 10 classes

(airplane, automobile, bird, Cat, deer, dog, frog, horse, ship, truck)

→ import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np

→ (X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()

X_train.shape — (50000, 32, 32, 3)

X_test.shape — (10000, 32, 32, 3)

X_train[0] — 3D array (32X32) - 3 channel.

plt.imshow(X_train[0]) — frog

[1] — Truck

plt.figure(figsize=(15,2))

→ Normalize Pixel Value (0-255 scale) ⟶ (0-1 scale)

→ ann = models.Sequential([ ])

ann.compile( )

ann.fit(X_train, y_train, epochs=5)

categorical_crossentropy V/S sparse_categorical_crossentropy :-

10 classes: airplane, automobile, bird, Cat, deer, dog, frog, horse, Ship, truck

loss = categorical_crossentropy

Y = [0 0 0 0 0 0 0 1 0] → One Hot Encoded form.

required.

loss = "sparse_categorical_crossentropy"

Y = 8 (value)

softmax — Normalize the Probabilities

of class1 = 0.45, class2 = 0.67 → Sigmoid

0.45/(0.45+0.67) = 0.40

0.67/(0.45+0.67) = 0.57

Sum = 1

Softmax (Normalize)